

Relazione – Laboratori di Data Science (Regressione Lineare e PCA)

Quarantani Diego

2026-04-01

Questa relazione raccoglie lo svolgimento degli esercizi del **Foglio 3 – Regressione Lineare e PCA** del corso di Data Science (Laurea in Fisica, Alma Mater Studiorum – Università di Bologna, a.a. 2025/2026). Per tutto il codice, si rimanda alla repository GitHub: [Laboratori-Data-Science](#)

1 Ampiezza della banda proibita in composti binari

1.1 Design del codice

L'analisi inizia con una **query** al database Materials Project, dalla quale vengono estratti tutti i composti binari insieme ai campi (**FIELDS**) di interesse:

```
1 with MPRester(API_KEY) as mpr:
2     docs = mpr.materials.summary.search(
3         num_elements=2,
4         fields=FIELDS,
5         chunk_size=1000,
6     )
```

I dati ottenuti vengono successivamente organizzati in due array: **X**, contenente le dieci variabili descrittive per ciascun materiale, e **Y**, contenente i corrispondenti valori di band gap. Per ottimizzare l'esecuzione del programma, entrambi gli array vengono salvati su disco tramite la libreria **pickle**, evitando di ripetere il download ad ogni esecuzione.

Successivamente il dataset viene suddiviso in un insieme di training (70%) e uno di test (30%) utilizzando la funzione **train_test_split** di **scikit-learn**:

```
1 X_train, X_test, Y_train, Y_test = train_test_split(
2     X, Y, test_size=0.3, random_state=42
3 )
```

Per rendere il codice più modulare e gestibile, i diversi modelli di regressione vengono implementati tramite delle **Pipeline**, che permettono di concatenare in modo ordinato le fasi di preprocessing e di apprendimento del modello:

```
1 pipelines = [
2     ('Linear regression',
3     Pipeline([('scaler', StandardScaler()), ('model', LinearRegression())])),
4     ('Ridge linear regression',
5     Pipeline([('scaler', StandardScaler()), ('model', Ridge(alpha=475.0))])),
6     ('Lasso linear regression',
7     Pipeline([('scaler', StandardScaler()), ('model', Lasso(alpha=0.006))])),
8 ]
```

Viene poi gestita separatamente la pipeline per la regressione Polinomiale:

```
1 last_pipe = Pipeline(
2     [
3         ('poli', PolynomialFeatures(degree=3, include_bias=False)),
4         ('scaler', StandardScaler()),
5         ('model', Ridge(alpha=110.0)),
6     ]
7 )
```

Per ciascuna pipeline poi avviene:

- **Pre-processing dei dati:** Inizialmente, viene applicata una standardizzazione delle feature mediante l'operatore `StandardScaler`. Tale procedura trasforma il vettore delle caratteristiche affinché presenti media nulla ($\mu = 0$) e varianza unitaria ($\sigma^2 = 1$). Questo passaggio risulta cruciale per uniformare l'apporto di ciascuna variabile al processo di ottimizzazione.
- **Addestramento del modello:** Si procede al fit del predittore all'interno della pipeline utilizzando il set di training. Mentre per i modelli lineari, Ridge e Polinomiali è possibile determinare i coefficienti tramite una soluzione analitica in forma chiusa, per il modello Lasso è necessario ricorrere ad algoritmi iterativi (discesa del gradiente). Si rimanda alla sezione 1.2 per l'analisi dettagliata relativa alla scelta del parametro di regolarizzazione λ .
- **Valutazione delle prestazioni:** Infine, vengono estratte e analizzate le metriche prestazionali per valutare i modelli:
 - Mean Squared Error (MSE): funzione di costo comunemente utilizzata nel modello *Linear Regression*. Nei modelli regolarizzati, come *Ridge* e *LASSO*, alla funzione costo si aggiunge rispettivamente una penalizzazione L_2 o L_1 .
 - Root Mean Squared Error (RMSE): radice quadrata del *Mean Squared Error*, utilizzata per esprimere l'errore medio di predizione da associare al singolo valore predetto.
 - R^2 Score: si utilizza l' R^2 score (coefficiente di determinazione). Calcolato tramite la funzione `r2_score` di `sklearn.metrics`, a valore tra 0 e 1, esso rappresenta la proporzione di varianza dei dati spiegata dal modello rispetto a un modello baseline (la media), in altre parole più il valore tende ad 1 più il modello ha predizioni accurate dato per dato e non solo in media. È definito come:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (1)$$

Dove:

- * y_i sono i valori osservati;
- * $\hat{y}_i$ sono i valori predetti dal modello;
- * $\bar{y}$ è la media dei valori osservati;
- * n è il numero totale di osservazioni.

Correlazione delle feature

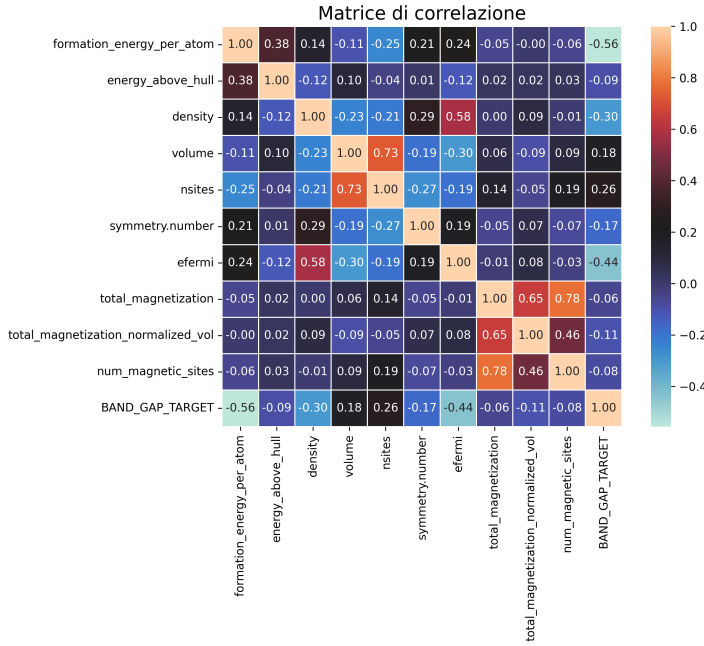


Figura 1: Correlazione lineare tra le 10 feature del set di addestramento e il band gap del materiale.

Per analizzare la relazione tra le feature e la variabile target, viene calcolata la matrice di correlazione di Pearson tramite `pandas.DataFrame.corr()`. In particolare in immagine 1 si riportano i coefficienti di correlazione rispetto al band gap e alle altre feature.

Si osserva come la feature maggiormente correlata con il band gap sia l'energia di formazione per atomo, che presenta una correlazione negativa significativa. Anche il livello di Fermi mostra una correlazione rilevante.

1.2 Regulation path

Per la selezione del parametro di regolarizzazione λ (chiamato *alpha* dalla libreria scikit-learn) nei diversi modelli, è stata adottata una procedura progressiva. Si è inizialmente scelto un valore elevato di λ , corrispondente a una forte regolarizzazione, e lo si è poi ridotto gradualmente. A ogni passo sono stati valutati l'errore sul set di test e il coefficiente di determinazione R^2 , individuando il valore ottimale in corrispondenza della diminuzione massima del MSE sul dataset di test.

1.3 Risultati

Metriche dei modelli

Tabella 1: Confronto delle prestazioni tra regressione lineare, Ridge, Lasso e regressione polinomiale di grado 3 con regolarizzazione Ridge. Per ragioni di spazio, i 285 coefficienti associati al modello polinomiale non sono mostrati.

Parametro	Linear	Ridge	Lasso	Polinomial Ridge d=3
MSE Train	0.7638	0.7638	0.7849	0.3898
MSE Test	0.7674	0.7674	0.7834	0.4232
RMSE Test	0.8760	0.8760	0.8851	0.6505
R ² Score Test	0.4246	0.4246	0.4126	0.6827
formation_energy_per_atom	-0.5861	-0.5861	-0.5245	
energy_above_hull	0.0926	0.0925	0.0073	
density	-0.0437	-0.0437	-0.0241	
volume	-0.1249	-0.1249	0	
nsites	0.2178	0.2178	0.0814	
symmetry.number	0.0268	0.0267	0	
efermi	-0.3467	-0.3467	-0.3174	
total_magnetization	0.0265	0.0265	0	
total_magnetization_normalized_vol	-0.0373	-0.0373	0	
num_magnetic_sites	-0.1692	-0.1691	-0.0957	

Analisi degli errori per range di gap

Tabella 2: Confronto dell'MSE sui diversi intervalli di band gap per i modelli considerati.

Range Band Gap (eV)	N. Campioni	Linear	Ridge	Lasso	Polinomiale
0.0	4814	0.3207	0.3092	0.3141	0.1497
(0.0, 1.0]	555	0.7984	0.7527	0.7744	0.5391
(1.0, 2.0]	315	0.6023	0.5853	0.5988	0.7179
(2.0, 3.0]	221	1.5728	1.5783	1.5751	1.4082
(3.0, 4.0]	120	3.2205	3.3089	3.2824	2.1946
(4.0, 5.0]	78	6.5713	6.7637	6.7057	4.3066
(5.0, 6.0]	100	8.7161	9.0825	8.9387	2.6189
(6.0, 7.0]	14	18.6694	19.0892	18.9314	11.6201
(7.0, 8.0]	10	19.8020	20.5937	20.2192	4.8241

Dalla Tabella 2 si osserva come i modelli lineari presentino un errore contenuto per materiali a basso band gap, mentre l'MSE cresce rapidamente per valori elevati. Ciò indica una difficoltà nel descrivere il comportamento non lineare ad ampio gap. La regressione polinomiale riduce significativamente l'errore nelle regioni ad alto band gap, suggerendo una migliore capacità di modellare relazioni non lineari tra le feature e la variabile target.

Analisi della Regolarizzazione e Generalizzazione

L'impatto delle tecniche di regolarizzazione sulla capacità di generalizzazione del modello è stato analizzato confrontando i risultati di Ridge, Lasso e della regressione polinomiale:

- **Regolarizzazione Ridge:** Applicando un parametro di penalizzazione $\alpha = 475.0$ (identificato come ottimale lungo il *regularization path*), non si osservano miglioramenti significativi nella capacità di generalizzazione. Le metriche prestazionali risultano identiche a quelle del modello lineare non regolarizzato; si osserva inoltre che i coefficienti non sono stati modificati dalla regolarizzazione, indicando che, durante la selezione del parametro, il modello migliore è risultato essere quello non regolarizzato.

- **Regolarizzazione Lasso:** L'impiego di Lasso con $\alpha = 6 \cdot 10^{-3}$ produce un peggioramento dell'errore sia sul set di *training* che su quello di *test*. Tale decremento prestazionale è riconducibile all'eccessiva semplificazione del modello (*feature selection* intrinseca), che ha azzerato i coefficienti relativi a variabili quali `volume`, `symmetry.number`, `total_magnetization` e `total_magnetization_normalized_vol`, inducendo un fenomeno di *underfitting*. In questo caso non è stato scelto il coefficiente α ottimale che avrebbe, come nel caso precedente, lasciato il modello invariato. La scelta è stata invece effettuata per mostrare come agisce la regolarizzazione LASSO, azzerando un numero n di coefficienti.
- **Regressione Polinomiale (grado 3) e Ridge:** L'introduzione di termini polinomiali, combinata con una regolarizzazione Ridge, ha prodotto il miglioramento più significativo. L'approccio ha permesso di catturare relazioni non lineari nei dati, portando lo scarto quadratico medio (*RMSE*) dal 88% al 65% sul dataset di test e incrementando di 0.2581 il coefficiente di determinazione R^2 .

2 Metodo PCA per la banca dati MNIST

2.1 Design del codice

Utilizzando le funzionalità della libreria scikit-learn, il dataset viene acquisito e successivamente ripartito nei sottoinsiemi di addestramento (train set) e di verifica (test set)

```
1 X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y, test_size=10000, train_size=50000, random_state=42
5 )
```

Viene creata una pipeline per la Regressione logistica:

```
1 mnist_pipeline = Pipeline(
2     [
3         ('scaler', StandardScaler()),
4         ('model', LogisticRegression(solver='sag', max_iter=100000, tol=0.001,
5             random_state=42)),
6     ]
7 )
8 mnist_pipeline.fit(X_train, y_train)
```

Viene utilizzata la funzione `accuracy_score` per la validazione delle predizioni e di due funzioni implementati manualmente per misurare il grado di sparsità dei coefficienti appresi:

```
1 def sparsity(weights):
2     return np.mean(weights == 0)
3
4 def sparsity_tol(weights, tol=1e-4):
5     return np.mean(np.isclose(weights, 0, atol=tol))
```

2.2 Risultati

Tabella 3: Valutazione delle prestazioni del modello: confronto tra la sparsità assoluta, la sparsità calcolata con tolleranza (10^{-4}) e l'accuratezza complessiva sul test set.

Sparsità	0.09566
Sparsità con tolleranza	0.09694
Accuratezza	0.9192

Come si evince dalla Tabella 3, il modello presenta una sparsità (intesa, come da definizione, quale percentuale di pesi non nulli) pari a circa lo 0.095. Questo indica che oltre il 90% dei coefficienti appresi è pari a zero. Ciò significa che la maggioranza dei pixel non contribuisce alla discriminazione del numero rappresentato; si tratta prevalentemente dei pixel situati ai bordi dell'immagine che costituiscono lo

sfondo nero costante. Nella visualizzazione successiva, questi pesi nulli corrispondono alle aree neutre (colorate in giallo). Sul fronte delle prestazioni predittive, il modello dimostra una buona efficacia, raggiungendo un'accuratezza del 91.92% sui dati di test.

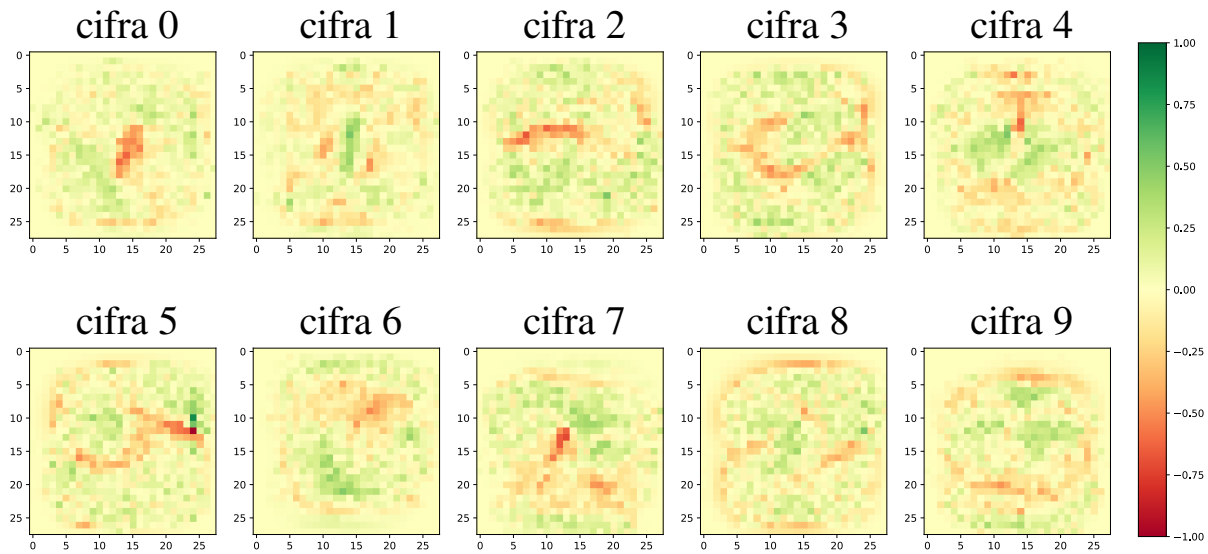


Figura 2: Mappe di calore dei pesi appresi dal modello di regressione logistica per ciascuna delle 10 classi (cifre 0-9) del dataset MNIST. Le intensità dei pixel rappresentano l'importanza di ciascuna area per la classificazione della relativa cifra.

L'analisi visiva dei pesi riportata in Figura 2 permette di interpretare le caratteristiche spaziali apprese dall'algoritmo per distinguere le diverse cifre. Si osserva, ad esempio, che per la classe corrispondente allo "0", l'area centrale presenta coefficienti negativi di elevata magnitudo. Questo comportamento è coerente con la morfologia della cifra, tipicamente caratterizzata da uno spazio vuoto al centro: l'attivazione di pixel in quell'area penalizza fortemente la probabilità che l'immagine rappresenti un "0". Si precisa infine che, al fine di permettere una corretta visualizzazione grafica, i valori mostrati nell'immagine sono stati normalizzati tra -1 e 1 scalandoli rispetto al coefficiente massimo assoluto, sebbene i pesi effettivi del modello varino su range differenti.

Analisi delle Componenti Principali (PCA)

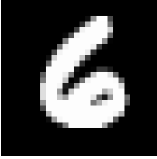
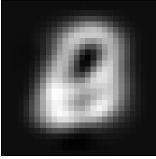
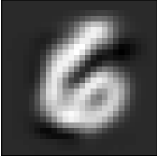
Tabella 4: Analisi della varianza spiegata in base al numero di componenti

N. Componenti	Varianza
5	0.3339
7	0.4095
10	0.4893
14	0.5649
20	0.6449
29	0.7249
42	0.7962
61	0.8558
87	0.9006
125	0.9348
179	0.9601
256	0.9793
366	0.9934
524	0.9996
750	1.0000

L'applicazione della PCA ha permesso di ridurre la dimensionalità del dataset da 784 caratteristiche originali a un numero inferiore di componenti principali, mantenendo comunque la struttura essenziale

delle cifre. La Tabella 5 illustra visivamente come la qualità della ricostruzione migliori all’aumentare della varianza spiegata accumulata dalle prime k componenti.

Tabella 5: Confronto tra immagini originali e ricostruzioni tramite PCA al variare del numero di componenti (k). Per ogni riga è indicata la varianza spiegata cumulata.

Digit	Originale	$k = 5$	$k = 10$	$k = 29$	$k = 125$	$k = 256$
3						
Varianza	100%	33.4%	48.9%	72.5%	93.5%	97.9%
6						
Varianza	100%	33.4%	48.9%	72.5%	93.5%	97.9%

Dalla tabella si osserva che già con $k = 125$ (meno del 15% delle dimensioni originali), l’immagine è chiaramente riconoscibile, a conferma del fatto che gran parte dell’informazione è contenuta nelle prime componenti principali.

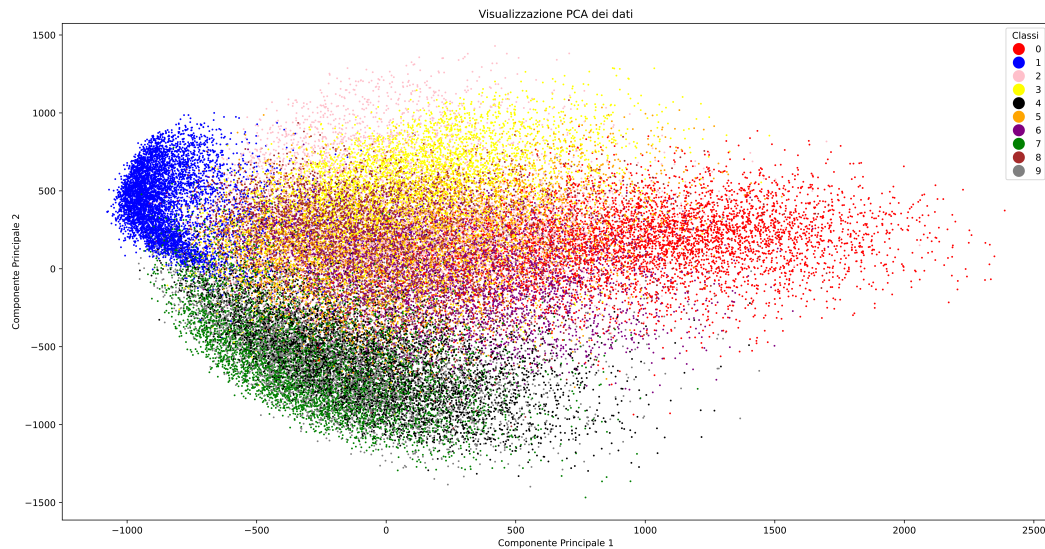


Figura 3: Proiezione bidimensionale del dataset MNIST tramite PCA. Ogni punto rappresenta un’immagine del dataset, colorata in base alla classe di appartenenza (0-9). La visualizzazione evidenzia come le prime due componenti principali riescano a catturare differenze morfologiche macroscopiche tra alcune classi.

L’analisi della Figura 3 rivela informazioni fondamentali sulla struttura del dataset. Si nota come la classe corrispondente alla cifra “1” (in blu) formi un cluster relativamente distinto sul lato sinistro del grafico, suggerendo che la sua forma lineare sia facilmente distinguibile dalle altre componenti. Al contrario, la regione centrale del grafico presenta una marcata sovrapposizione tra diverse classi, come il “2” (rosa), il “6” (viola) e l’ “8” (marrone); tale fenomeno è dovuto alla somiglianza strutturale di queste cifre, che condividono molti pixel attivi nelle stesse posizioni spaziali.

In sintesi, la PCA in due dimensioni permetta di individuare dei macro-agglomerati, essa non è sufficiente per una separazione lineare delle 10 classi. Questo giustifica l’utilizzo di un numero più elevato

di componenti principali per l'addestramento dei modelli di classificazione, al fine di preservare una quota di varianza utile a distinguere le cifre morfologicamente simili.

Performance

Il processo di training, inizialmente attestato su 5:22 minuti per lo spazio delle feature originale a 784 dimensioni, è stato ridotto a circa 10 secondi previo filtraggio dei dati tramite PCA ($d=100$). Il modello ha garantito un'accuratezza del 91.66% (con uno scarto del 2.6 % rispetto al modello originale).

3 Auto-facce e algoritmo PCA

3.1 Design del codice

Il primo passo per l'estrazione delle *eigenfaces* (auto-facce) consiste nel caricamento e nel pre-processing del dataset. Le immagini vettorializzate vengono standardizzate affinché abbiano media nulla e varianza unitaria tramite `StandardScaler`:

```
1 st_instance = StandardScaler()
2 faces_scaled = st_instance.fit_transform(faces)
3 X_train = faces_scaled
```

Successivamente, viene istanziato e addestrato il modello PCA sull'intero training set per calcolare le componenti principali, le quali rappresentano le direzioni di massima varianza nello spazio delle immagini:

```
1 pca = PCA(random_state=42)
2 pca.fit(X_train)
```

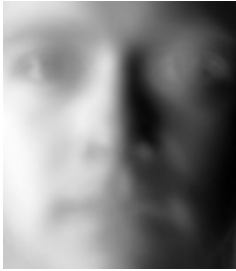
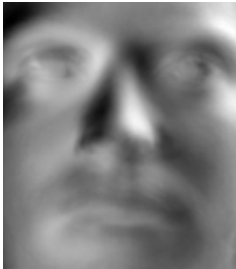
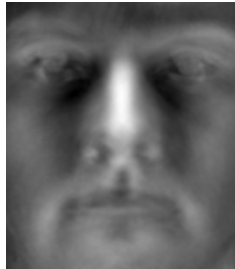
Per testare la capacità di generalizzazione e di ricostruzione del modello, sono state caricate e pre-processate due immagini esterne (una foto personale e l'immagine di una tigre). Queste immagini vengono prima proiettate nello spazio latente ridotto e poi ricostruite utilizzando un numero progressivamente crescente di componenti k :

```
1 for k in range(5, len(img_pca)):
2     img_pca_filtered = img_pca.copy()
3
4     # Azzera tutte le componenti successive a k
5     img_pca_filtered[k:] = 0
6
7     img_restored = pca.inverse_transform(img_pca_filtered)
8     img_restored_original_scale = st_instance.inverse_transform(img_restored)
```

Durante questo processo iterativo, viene calcolato l'Errore Quadratico Medio (MSE) tra l'immagine originale e quella ricostruita, per quantificare numericamente la qualità della ricostruzione al variare di k .

3.2 Risultati

Tabella 6: Visualizzazione delle prime 15 autofacce (Eigenfaces) estratte tramite PCA.

EF 0	EF 1	EF 2	EF 3	EF 4
				
EF 5	EF 6	EF 7	EF 8	EF 9
				
EF 10	EF 11	EF 12	EF 13	EF 14
				

Come negli altri casi, le autofacce rappresentano le direzioni di maggiore varianza all'interno del dataset, cioè i tratti che più subiscono variazioni (e che quindi racchiudono maggiore informazione) tra le varie immagini. Si nota che le prime autofacce, in particolare le prime 5, contengono l'informazione sulla distribuzione della luce nell'immagine (ad es. EF 0: luce da destra, EF 1: luce da sinistra, EF 2: luce da sopra).

Utilizzando una foto esterna (Tabella 7), la composizione con poche autofacce è abbastanza soddisfacente: i tratti della faccia originale iniziano a delinearsi intorno a $k = 490$ e diventano ben riconoscibili superate le 1000 componenti.

Il limite della proiezione su base PCA diventa palese introducendo l'immagine di una tigre (Tabella 8). Le *eigenfaces* costituiscono uno spazio latente ottimizzato esclusivamente per i volti umani; di conseguenza, la ricostruzione di un soggetto che non condivide alcuna caratteristica strutturale (occhi, naso, bocca nelle tipiche posizioni umane) è impossibile (le 2410 informazioni selezionate per la descrizione di un volto non sono ovviamente sufficienti per la descrizione di una qualsiasi soggetto in uno spazio 32256-dimensionale).

Tabella 7: Ricostruzione di un'immagine fuori distribuzione (*Out-of-Distribution*): Diego.

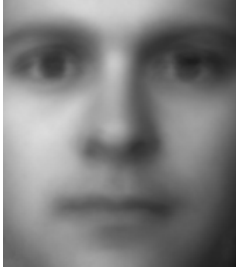
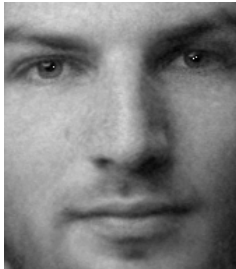
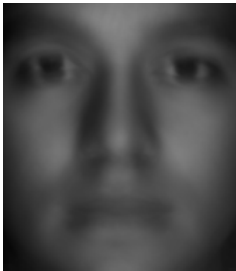
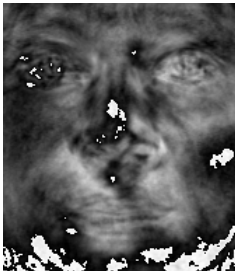
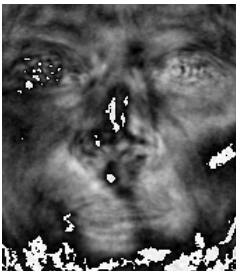
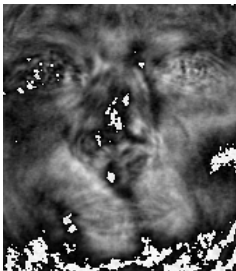
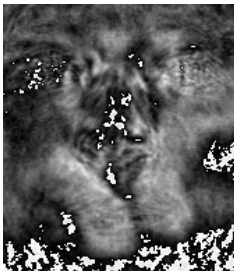
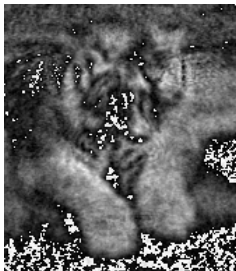
Originale	$k = 5$	$k = 45$	$k = 166$	$k = 348$
				
$k = 490$	$k = 631$	$k = 1055$	$k = 2410$	
				

Tabella 8: Ricostruzione di un'immagine fuori distribuzione (*Out-of-Distribution*): una tigre. I vettori di base, essendo stati appresi su volti umani, faticano a ricostruire pattern completamente differenti, originando evidenti artefatti.

Originale	$k = 5$	$k = 45$	$k = 166$	$k = 348$
				
$k = 490$	$k = 631$	$k = 1055$	$k = 2410$	
				

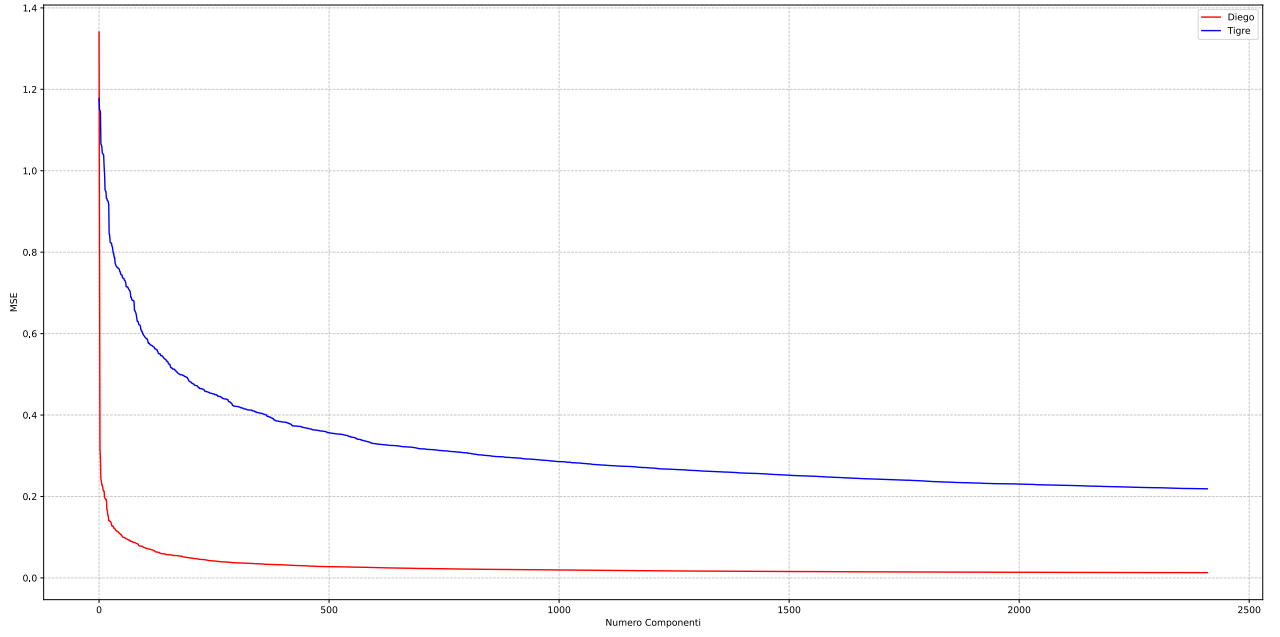


Figura 4: Andamento dell'Errore Quadratico Medio (MSE) in funzione del numero di componenti utilizzate (k). In rosso l'andamento dell'immagine del sottoscritto, che raggiunge un valore minimo di $er_F = 0.01292$; in blu l'andamento dell'immagine della tigre $er_T = 0.2188$

Infine, l'analisi dell'Errore Quadratico Medio (Figura 4) conferma quantitativamente le osservazioni empiriche: l'errore relativo all'immagine della tigre si riduce molto più lentamente e si stabilizza su un valore di *differenza dall'immagine originale* circa 15 volte superiore rispetto a quello dell'immagine del volto. Al contrario, quest'ultima mostra una decrescita molto più marcata già nelle prime 250 autofacce.